

Web Development Notes

HTML • CSS • JavaScript

Basic to Intermediate Level

HTML

CSS

JavaScript

HTML — HyperText Markup Language

1. What is HTML?

HTML is the standard markup language used to create web pages. It describes the structure of a webpage using a series of elements (tags). Browsers read HTML and render it as visual content on screen.

NOTE: HTML is NOT a programming language — it is a markup language that structures content.

Basic HTML Document Structure

Boilerplate HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My First Page</title>
  </head>
  <body>
    <h1>Hello, World!</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

- `<!DOCTYPE html>` — Tells browser this is HTML5
- `<html>` — Root element of the page
- `<head>` — Contains meta info (not visible)
- `<body>` — Contains all visible content

2. HTML Elements & Tags

An HTML element has an opening tag, content, and a closing tag:

Element Structure

```
<tagname>Content goes here</tagname>
<p>This is a paragraph</p>
```

Some elements are self-closing (void elements) and need no closing tag:

Void Elements

```
<br>
<hr>

<input type="text">
```

3. Headings & Paragraphs

Headings & Text

```
<h1>Heading 1 (Largest)</h1>
<h2>Heading 2</h2>
```

```
<h3>Heading 3</h3>
<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6 (Smallest)</h6>
<p>This is a paragraph of text.</p>
<br>  <!-- Line break -->
<hr>  <!-- Horizontal rule -->
```

NOTE: Use only ONE <h1> per page for SEO and accessibility.

4. Text Formatting Tags

```
Text Tags
<b>Bold text</b>          <!-- visual bold -->
<strong>Important!</strong> <!-- semantic bold -->
<i>Italic text</i>        <!-- visual italic -->
<em>Emphasized</em>      <!-- semantic italic -->
<u>Underlined</u>
<s>Strikethrough</s>
<mark>Highlighted</mark>
<small>Smaller text</small>
<sup>Superscript</sup>   e.g. x<sup>2</sup>
<sub>Subscript</sub>     e.g. H<sub>2</sub>O
<code>Inline code</code>
<pre>Preformatted text block</pre>
<blockquote>A quoted block of text</blockquote>
```

5. Links & Images

Anchor Tag (Links)

```
Links
<!-- External link -->
<a href="https://google.com">Visit Google</a>

<!-- Open in new tab -->
<a href="https://google.com" target="_blank">Open in new tab</a>

<!-- Link to email -->
<a href="mailto:hello@example.com">Email Me</a>

<!-- Internal page link -->
<a href="#section1">Go to Section 1</a>
<h2 id="section1">Section 1</h2>
```

Images

```
Images


<!-- With width & height -->

```

| **NOTE:** Always add the alt attribute for accessibility and SEO.

6. Lists

Lists

```
<!-- Unordered List (bullets) -->
<ul>
  <li>Apple</li>
  <li>Banana</li>
  <li>Cherry</li>
</ul>

<!-- Ordered List (numbered) -->
<ol>
  <li>First step</li>
  <li>Second step</li>
  <li>Third step</li>
</ol>

<!-- Description List -->
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language</dd>
  <dt>CSS</dt>
  <dd>Cascading Style Sheets</dd>
</dl>
```

7. Tables

HTML Table

```
<table>
  <thead>
    <tr>
      <th>Name</th>
      <th>Age</th>
      <th>City</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Alice</td>
      <td>25</td>
      <td>Delhi</td>
    </tr>
    <tr>
      <td>Bob</td>
      <td>30</td>
      <td>Mumbai</td>
    </tr>
  </tbody>
</table>
```

- tr — Table Row
- th — Table Header (bold by default)
- td — Table Data (regular cell)
- thead / tbody — Semantic grouping of rows

8. HTML Forms

HTML Form Example

```
<form action="/submit" method="POST">

  <!-- Text Input -->
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" placeholder="Enter name">

  <!-- Email -->
  <input type="email" name="email" required>

  <!-- Password -->
  <input type="password" name="pass">

  <!-- Number -->
  <input type="number" min="1" max="100">

  <!-- Radio Buttons -->
  <input type="radio" name="gender" value="male"> Male
  <input type="radio" name="gender" value="female"> Female

  <!-- Checkbox -->
  <input type="checkbox" name="agree" value="yes"> I Agree

  <!-- Dropdown -->
  <select name="city">
    <option value="delhi">Delhi</option>
    <option value="mumbai">Mumbai</option>
  </select>

  <!-- Textarea -->
  <textarea name="msg" rows="4" cols="40"></textarea>

  <!-- Submit Button -->
  <button type="submit">Submit</button>
</form>
```

9. Semantic HTML (Intermediate)

Semantic elements clearly describe their meaning to both the browser and developer.

Semantic Tags

```
<header>    <!-- Top section / navigation area -->
<nav>       <!-- Navigation links -->
<main>      <!-- Main content area -->
<section>   <!-- A thematic section -->
<article>   <!-- Independent content (blog post, etc.) -->
<aside>     <!-- Sidebar content -->
<footer>    <!-- Bottom section -->
<figure>    <!-- Image with caption -->
  
  <figcaption>Caption here</figcaption>
</figure>
<details>   <!-- Collapsible content -->
  <summary>Click to expand</summary>
  <p>Hidden content here</p>
</details>
```

NOTE: Prefer semantic tags over generic `<div>` and `<span>` for better SEO and accessibility.

10. HTML Attributes

Attributes provide extra information about an element and go inside the opening tag.

Attributes

```
<!-- Common Attributes -->
id="myId"           <!-- Unique identifier -->
class="myClass"     <!-- For CSS targeting -->
style="color:red"   <!-- Inline CSS -->
title="Tooltip"     <!-- Hover tooltip -->
hidden              <!-- Hides element -->
disabled            <!-- Disables input -->
required            <!-- Makes field mandatory -->
readonly            <!-- Input is read-only -->
data-info="value"   <!-- Custom data attributes -->
```

CSS — Cascading Style Sheets

1. What is CSS?

CSS is used to style and layout web pages — controlling colors, fonts, spacing, positioning, and more. CSS is applied to HTML elements using selectors and property-value pairs.

Ways to Apply CSS

Applying CSS

```
<!-- 1. Inline CSS -->
<p style="color: red; font-size: 18px;">Hello</p>

<!-- 2. Internal CSS (in <head>) -->
<style>
  p { color: blue; }
</style>

<!-- 3. External CSS (recommended) -->
<link rel="stylesheet" href="styles.css">
```

NOTE: Always use external CSS for real projects. It keeps HTML clean and CSS reusable.

2. CSS Selectors

CSS Selectors

```
/* Element Selector */
p { color: black; }

/* Class Selector (.) */
.card { background: white; }

/* ID Selector (#) */
#header { height: 80px; }

/* Group Selector */
h1, h2, h3 { font-family: Arial; }

/* Descendant Selector */
nav a { text-decoration: none; }

/* Child Selector (>) */
ul > li { color: gray; }

/* Pseudo-class */
a:hover { color: red; }
li:nth-child(2) { background: yellow; }
input:focus { border: 2px solid blue; }

/* Pseudo-element */
p::first-line { font-weight: bold; }
h1::before { content: "→ "; }

/* Attribute Selector */
```

```
input[type="email"] { border: 1px solid green; }

/* Universal Selector */
* { box-sizing: border-box; margin: 0; padding: 0; }
```

3. Colors & Units

Colors & Units

```
/* Colors */
color: red; /* Named color */
color: #FF5733; /* Hex */
color: rgb(255, 87, 51); /* RGB */
color: rgba(255, 87, 51, 0.5); /* RGBA (with opacity) */
color: hsl(9, 100%, 60%); /* HSL */

/* Units */
font-size: 16px; /* Pixels (absolute) */
font-size: 1.5em; /* Relative to parent font-size */
font-size: 1.5rem; /* Relative to root font-size */
width: 50%; /* Percentage of parent */
height: 100vh; /* Viewport height */
width: 100vw; /* Viewport width */
```

4. The Box Model

Every HTML element is a box. The box model defines the space an element takes up.

Box Model

```
div {
  width: 300px; /* Content width */
  padding: 20px; /* Space INSIDE border */
  border: 2px solid; /* Border around padding */
  margin: 30px; /* Space OUTSIDE border */
}

/* Individual sides */
padding: 10px 20px 10px 20px; /* top right bottom left */
margin: 10px 20px; /* top-bottom left-right */

/* Fix total sizing (recommended) */
* { box-sizing: border-box; }
```

NOTE: With box-sizing: border-box, width includes padding and border, preventing layout surprises.

5. Typography & Text

Typography

```
p {
  font-family: 'Arial', sans-serif;
  font-size: 18px;
  font-weight: bold; /* normal | bold | 100-900 */
  font-style: italic;
  line-height: 1.6; /* Spacing between lines */
  letter-spacing: 1px;
```



```

text-align: center;    /* left | right | justify */
text-decoration: underline;
text-transform: uppercase; /* lowercase | capitalize */
color: #333;
}

/* Google Fonts (add in <head>) */
<link href="https://fonts.googleapis.com/css2?family=Roboto&display=swap"
rel="stylesheet">
body { font-family: 'Roboto', sans-serif; }

```

6. Backgrounds

Backgrounds

```

div {
  background-color: #f0f0f0;
  background-image: url('bg.jpg');
  background-size: cover;    /* contain | auto | 100% */
  background-position: center;
  background-repeat: no-repeat;
  background-attachment: fixed; /* Parallax effect */

  /* Shorthand */
  background: #fff url('bg.jpg') no-repeat center / cover;

  /* Gradient */
  background: linear-gradient(to right, #FF5733, #6C63FF);
  background: radial-gradient(circle, #fff, #000);
}

```

7. Display & Positioning

Display Property

Display Values

```

display: block;    /* Takes full width, new line */
display: inline;    /* Only takes content width */
display: inline-block; /* Inline but respects width/height */
display: none;    /* Hides element (removed from flow) */
display: flex;    /* Flexbox layout */
display: grid;    /* Grid layout */

```

Position Property

Positioning

```

/* Static (default) */
position: static;

/* Relative - moves from its normal position */
position: relative;
top: 10px; left: 20px;

/* Absolute - positioned relative to nearest positioned parent */

```

```
position: absolute;
top: 0; right: 0;

/* Fixed — stays fixed on screen when scrolling */
position: fixed;
top: 0; width: 100%;

/* Sticky — sticks when scrolled to */
position: sticky;
top: 0;
```

8. Flexbox (Intermediate)

Flexbox is a layout method for arranging items in a row or column with powerful alignment options.

```
Flexbox
/* Parent (Flex Container) */
.container {
  display: flex;
  flex-direction: row;          /* row | column | row-reverse */
  justify-content: center;      /* Main axis: flex-start | flex-end | space-between
| space-around */
  align-items: center;          /* Cross axis: stretch | flex-start | flex-end */
  flex-wrap: wrap;              /* Allow items to wrap */
  gap: 16px;                    /* Space between items */
}

/* Child (Flex Items) */
.item {
  flex: 1;                      /* Grow to fill space equally */
  flex-grow: 2;                 /* Grow twice as much as others */
  flex-shrink: 0;               /* Don't shrink */
  flex-basis: 200px;            /* Starting size */
  align-self: flex-end;         /* Override align-items for this item */
}
```

9. CSS Grid (Intermediate)

CSS Grid is a 2D layout system — it works with both rows and columns simultaneously.

```
CSS Grid
.grid {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr; /* 3 columns */
  grid-template-rows: auto;
  gap: 20px;

  /* Shorthand */
  grid-template-columns: repeat(3, 1fr);

  /* Responsive with auto-fill */
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
}

/* Placing items */
.item {
  grid-column: 1 / 3; /* Span from column 1 to 3 */
}
```

```
grid-row: 1 / 2;
}
```

10. Responsive Design & Media Queries

Media Queries

```
/* Mobile first approach */
body { font-size: 14px; }

/* Tablet: 768px and up */
@media (min-width: 768px) {
  body { font-size: 16px; }
}

/* Desktop: 1024px and up */
@media (min-width: 1024px) {
  body { font-size: 18px; }
  .container { max-width: 1200px; margin: 0 auto; }
}

/* Max-width (desktop first) */
@media (max-width: 600px) {
  nav { flex-direction: column; }
}
```

11. CSS Variables & Transitions

Variables & Transitions

```
/* Define CSS Variables */
:root {
  --primary: #6C63FF;
  --secondary: #FF6584;
  --font-size: 16px;
}

/* Use them */
button {
  background: var(--primary);
  font-size: var(--font-size);
}

/* Transitions (smooth changes) */
button {
  background: var(--primary);
  transition: background 0.3s ease, transform 0.2s;
}
button:hover {
  background: var(--secondary);
  transform: scale(1.05);
}

/* Keyframe Animations */
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(20px); }
  to { opacity: 1; transform: translateY(0); }
}
```

```
.card { animation: fadeIn 0.5s ease forwards; }
```

JavaScript

1. What is JavaScript?

JavaScript (JS) is a programming language that makes web pages interactive. It runs in the browser and can manipulate HTML/CSS, handle user events, fetch data, and much more.

| NOTE: JavaScript is case-sensitive: myVar and myvar are different variables.

2. Variables

```
Variables
// let - block-scoped, can be reassigned
let name = "Alice";
name = "Bob";    // OK

// const - block-scoped, cannot be reassigned
const PI = 3.14;
// PI = 3.15;    // ERROR

// var - function-scoped (older, avoid using)
var age = 25;

// Multiple assignment
let x = 1, y = 2, z = 3;
```

3. Data Types

```
Data Types
// Primitive Types
let str   = "Hello";           // String
let num   = 42;                // Number
let dec   = 3.14;              // Number (float)
let bool  = true;              // Boolean (true/false)
let empty = null;              // Null (intentionally empty)
let undef = undefined;         // Undefined (not assigned)
let big   = 9007199254740991n; // BigInt

// Reference Types
let arr = [1, 2, 3];           // Array
let obj = { name: "Alice" };   // Object
let fn  = function() {};       // Function

// Type checking
typeof "Hello" // 'string'
typeof 42      // 'number'
typeof true    // 'boolean'
typeof null    // 'object' (historical quirk!)
```

4. Operators

Operators

```
// Arithmetic
5 + 3    // 8
10 - 4   // 6
3 * 4    // 12
15 / 4   // 3.75
15 % 4   // 3 (remainder)
2 ** 3   // 8 (power)

// Assignment
let x = 10;
x += 5;  // x = 15
x -= 3;  // x = 12
x *= 2;  // x = 24
x++;    // x = 25 (increment)
x--;    // x = 24 (decrement)

// Comparison
5 == '5'    // true (type coercion)
5 === '5'   // false (strict: type + value)
5 !== 6     // true
10 > 5      // true
10 <= 10    // true

// Logical
true && false // false (AND)
true || false // true (OR)
!true        // false (NOT)

// Ternary
let result = age >= 18 ? "Adult" : "Minor";
```

NOTE: Always use `===` (strict equality) instead of `==` to avoid unexpected type coercion bugs.

5. Strings

String Methods

```
let name = "Alice";

// String methods
name.length           // 5
name.toUpperCase()    // 'ALICE'
name.toLowerCase()    // 'alice'
name.includes('li')   // true
name.startsWith('A')  // true
name.endsWith('e')    // true
name.indexOf('l')     // 2
name.slice(1, 3)      // 'li'
name.replace('A', 'B') // 'Blice'
name.trim()           // removes whitespace
name.split('')        // ['A', 'l', 'i', 'c', 'e']

// Template Literals (backticks) - very useful!
let age = 25;
let msg = `My name is ${name} and I am ${age} years old.`;
let multi = `Line 1
Line 2`;           // Multi-line string
```

6. Arrays

Arrays

```
let fruits = ['Apple', 'Banana', 'Cherry'];

// Access
fruits[0]      // 'Apple'
fruits.length  // 3

// Add / Remove
fruits.push('Mango');    // Add at end
fruits.pop();           // Remove from end
fruits.unshift('Grape'); // Add at start
fruits.shift();         // Remove from start
fruits.splice(1, 1);     // Remove at index 1

// Search
fruits.includes('Banana') // true
fruits.indexOf('Cherry')  // 2

// Iteration
fruits.forEach(fruit => console.log(fruit));

// Transform (returns new array)
let upper = fruits.map(f => f.toUpperCase());

// Filter
let long = fruits.filter(f => f.length > 5);

// Find first match
let found = fruits.find(f => f.startsWith('B'));

// Reduce to single value
let nums = [1, 2, 3, 4];
let sum = nums.reduce((acc, n) => acc + n, 0); // 10

// Sort
fruits.sort();
nums.sort((a, b) => a - b); // Numeric sort

// Join & Spread
fruits.join(', ');          // 'Apple, Banana, Cherry'
let combined = [...fruits, 'Mango']; // Spread
```

7. Objects

Objects

```
// Creating an object
let person = {
  name: 'Alice',
  age: 25,
  city: 'Delhi',
  greet: function() {
    return `Hi, I'm ${this.name}`;
  }
};
```

```
// Access properties
person.name // 'Alice' (dot notation)
person['age'] // 25 (bracket notation)

// Add / Update / Delete
person.email = 'a@b.com'; // Add
person.age = 26; // Update
delete person.city; // Delete

// Methods
Object.keys(person) // ['name', 'age', ...]
Object.values(person) // ['Alice', 26, ...]
Object.entries(person) // [['name', 'Alice'], ...]

// Destructuring
const { name, age } = person;
const { name: personName } = person; // Rename

// Spread operator
let copy = { ...person, country: 'India' };
```

8. Conditionals

Conditionals

```
// if / else if / else
let score = 75;

if (score >= 90) {
  console.log('A Grade');
} else if (score >= 75) {
  console.log('B Grade');
} else if (score >= 60) {
  console.log('C Grade');
} else {
  console.log('Fail');
}

// Switch
let day = 'Mon';
switch(day) {
  case 'Mon': console.log('Monday'); break;
  case 'Tue': console.log('Tuesday'); break;
  default: console.log('Other day');
}

// Nullish coalescing (??) – use default if null/undefined
let username = null;
let display = username ?? 'Guest'; // 'Guest'

// Optional chaining (?.) – safe property access
let user = null;
console.log(user?.name); // undefined (no error)
```

9. Loops

Loops

```
// for loop
for (let i = 0; i < 5; i++) {
  console.log(i); // 0, 1, 2, 3, 4
}

// while loop
let i = 0;
while (i < 3) {
  console.log(i);
  i++;
}

// do...while (runs at least once)
do {
  console.log(i);
  i++;
} while (i < 5);

// for...of (iterate array values)
let colors = ['red', 'green', 'blue'];
for (let color of colors) {
  console.log(color);
}

// for...in (iterate object keys)
let obj = { a: 1, b: 2 };
for (let key in obj) {
  console.log(key, obj[key]);
}

// break and continue
for (let i = 0; i < 10; i++) {
  if (i === 5) break; // Stop loop
  if (i % 2 === 0) continue; // Skip even
  console.log(i);
}
```

10. Functions

Functions

```
// Function Declaration
function greet(name) {
  return `Hello, ${name}!`;
}
greet('Alice'); // 'Hello, Alice!'

// Function Expression
const add = function(a, b) {
  return a + b;
};

// Arrow Function
const multiply = (a, b) => a * b;
const square = n => n * n; // Single param, no parens needed

// Default Parameters
function greet(name = 'Guest') {
  return `Hello, ${name}!`;
}
```

```

}
greet(); // 'Hello, Guest'
greet('Bob'); // 'Hello, Bob'

// Rest Parameters
function sum(...nums) {
  return nums.reduce((a, b) => a + b, 0);
}
sum(1, 2, 3, 4); // 10

// Higher-order functions (functions that take/return functions)
function repeat(fn, times) {
  for (let i = 0; i < times; i++) fn();
}
repeat(() => console.log('Hi'), 3);

```

11. DOM Manipulation

The DOM (Document Object Model) is the browser's tree representation of your HTML. JavaScript can read and modify it.

DOM Manipulation

```

// Selecting Elements
document.getElementById('myId');
document.querySelector('.myClass'); // First match
document.querySelectorAll('p'); // All <p> tags
document.getElementsByClassName('btn'); // HTMLCollection

// Changing Content
let el = document.querySelector('h1');
el.textContent = 'New Heading'; // Text only
el.innerHTML = '<strong>Bold!</strong>'; // HTML string

// Changing Styles
el.style.color = 'red';
el.style.fontSize = '24px';

// Class manipulation
el.classList.add('active');
el.classList.remove('hidden');
el.classList.toggle('dark');
el.classList.contains('active'); // true/false

// Attributes
el.getAttribute('href');
el.setAttribute('href', '#top');
el.removeAttribute('disabled');

// Creating Elements
let div = document.createElement('div');
div.textContent = 'I am new!';
document.body.appendChild(div);
document.body.removeChild(div);

// Traversal
el.parentElement
el.children
el.nextElementSibling
el.previousElementSibling

```

12. Events

Events

```
let btn = document.querySelector('#myBtn');

// addEventListener (recommended)
btn.addEventListener('click', function(event) {
  console.log('Clicked!', event);
  event.preventDefault(); // Prevent default action
  event.stopPropagation(); // Stop event bubbling
});

// Common Events
// click, dblclick, mouseover, mouseout, mousemove
// keydown, keyup, keypress
// submit, change, input, focus, blur
// load, scroll, resize

// Event on input
let input = document.querySelector('input');
input.addEventListener('input', (e) => {
  console.log('Value:', e.target.value);
});

// Event Delegation (handle events on parent)
document.querySelector('ul').addEventListener('click', (e) => {
  if (e.target.tagName === 'LI') {
    console.log('Clicked item:', e.target.textContent);
  }
});
```

13. Asynchronous JavaScript (Intermediate)

Callbacks → Promises → Async/Await

Async JavaScript

```
// 1. Callback (old style)
setTimeout(() => {
  console.log('Runs after 2 seconds');
}, 2000);

// 2. Promises
const fetchData = new Promise((resolve, reject) => {
  let success = true;
  if (success) resolve('Data loaded');
  else reject('Error!');
});

fetchData
  .then(data => console.log(data))
  .catch(err => console.error(err))
  .finally(() => console.log('Done'));

// 3. Async / Await (cleanest way)
```

```

async function getData() {
  try {
    let response = await fetch('https://api.example.com/data');
    let json = await response.json();
    console.log(json);
  } catch (error) {
    console.error('Failed:', error);
  }
}
getData();

```

14. Fetch API (AJAX)

Fetch API

```

// GET request
async function getUsers() {
  const res = await fetch('https://jsonplaceholder.typicode.com/users');
  const users = await res.json();
  users.forEach(u => console.log(u.name));
}

// POST request
async function createPost() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title: 'Hello', body: 'World', userId: 1 })
  });
  const data = await res.json();
  console.log(data);
}

```

15. Local Storage & Session Storage

Web Storage

```

// localStorage - persists even after browser closes
localStorage.setItem('username', 'Alice');
localStorage.getItem('username'); // 'Alice'
localStorage.removeItem('username');
localStorage.clear(); // Remove all

// Store objects (must JSON stringify)
let user = { name: 'Alice', age: 25 };
localStorage.setItem('user', JSON.stringify(user));
let saved = JSON.parse(localStorage.getItem('user'));

// sessionStorage - clears when tab closes
sessionStorage.setItem('token', 'abc123');
sessionStorage.getItem('token');

```

16. Modern JavaScript (ES6+)

ES6+ Features

```

// Destructuring
const [a, b, ...rest] = [1, 2, 3, 4, 5];
const { name, age, ...others } = person;

// Spread / Rest
const arr1 = [1, 2]; const arr2 = [3, 4];
const merged = [...arr1, ...arr2]; // [1,2,3,4]

// Short-circuit evaluation
const isLoggedIn = true;
isLoggedIn && console.log('Welcome!'); // Runs if true
const val = null || 'default'; // 'default'

// Modules (import/export)
// math.js
export const add = (a, b) => a + b;
export default function square(n) { return n * n; }

// main.js
import square, { add } from './math.js';

// Map & Set
const map = new Map();
map.set('key', 'value');
map.get('key'); // 'value'

const set = new Set([1, 2, 2, 3]); // {1, 2, 3} - no duplicates
set.has(2); // true

// Nullish coalescing & optional chaining
const x = data?.user?.name ?? 'Anonymous';

```

17. Error Handling

Error Handling

```

// try / catch / finally
try {
  let result = JSON.parse('invalid json');
} catch (error) {
  console.error('Error name:', error.name);
  console.error('Error message:', error.message);
} finally {
  console.log('This always runs');
}

// Throwing custom errors
function divide(a, b) {
  if (b === 0) throw new Error('Cannot divide by zero!');
  return a / b;
}

try {
  divide(10, 0);
} catch (e) {
  console.log(e.message); // 'Cannot divide by zero!'
}

```

Quick Reference Summary

Topic	Key Concept	Example
HTML Structure	DOCTYPE, html, head, body	<code><!DOCTYPE html><html>...</code>
HTML Tags	Opening & closing tags	<code><p>Text</p></code>
HTML Forms	input, select, textarea	<code><input type='email'></code>
Semantic HTML	header, main, article, footer	<code><main><article>...</code>
CSS Selectors	element, class, id, pseudo	<code>p, .card, #nav, a:hover</code>
Box Model	content, padding, border, margin	<code>padding: 10px; margin: 20px;</code>
Flexbox	display:flex, justify, align	<code>justify-content: space-between</code>
CSS Grid	display:grid, columns, rows	<code>grid-template-columns: 1fr 2fr</code>
Media Query	Responsive breakpoints	<code>@media (max-width: 768px){}</code>
JS Variables	let, const, var	<code>const PI = 3.14;</code>
JS Arrays	Methods for list data	<code>.map(), .filter(), .reduce()</code>
JS Objects	Key-value data structure	<code>{name: 'Alice', age: 25}</code>
JS DOM	Select & modify HTML elements	<code>querySelector, textContent</code>
JS Events	React to user actions	<code>addEventListener('click', fn)</code>
Async JS	Fetch/await for API calls	<code>await fetch(url)</code>

Happy Coding! 💎 Build something amazing.